



LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING



From package to postinstall payload: Inside the Mastra npm supply chain compromise by Sapphire Sleet

Threat Report

Classification	TLP:CLEAR
Published	2026-06-18
Version	1.0
Author	Lost Boys Cyber
Report Type	Threat Report

TLP:CLEAR | Lost Boys Cyber | From package to postinstall payload: Inside the Mastra npm supply chain compromise by Sapphire Sleet - Threat Report

Published: 2026-06-18 | TLP:CLEAR | Version: 1.0 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary
2. Intelligence Requirements
3. Source Review & Confidence
4. Threat Actor / Campaign Overview
5. Tactics, Techniques, and Procedures
6. Infrastructure and Indicators
7. Aviation / Aerospace Sector Impact
8. Detection Engineering
 - 8.1. Detection Logic Summary
9. Threat Hunting
 - 9.1. Hunt Hypothesis
10. Incident Response and Containment
11. MITRE ATT&CK Mapping
12. Recommended Actions
13. References

From package to postinstall payload: Inside the Mastra npm supply chain compromise by Sapphire Sleet

1. Executive Summary

Microsoft reported a Mastra npm supply-chain compromise attributed to Sapphire Sleet, describing poisoned npm packages and postinstall payload behavior affecting more than 140 projects in the Mastra AI ecosystem. The incident is relevant to defenders because package postinstall hooks execute during dependency installation and CI/CD builds, turning developer workstations and build runners into high-value collection points.

This report treats the event as a developer-supply-chain intrusion risk rather than a malware-analysis case: no sample was provided to this automation run, and no sandbox or reverse-engineering evidence was fabricated. Defensive priority should be dependency inventory, lockfile review, build-runner containment, secret rotation, and telemetry hunts for suspicious npm lifecycle script execution.

2. Intelligence Requirements

Requirement	Answer
Who is associated?	Microsoft attributes the activity to Sapphire Sleet; corroborating public commentary links Sapphire Sleet to North Korean activity, but Microsoft is the primary source used here.
What was targeted?	Mastra AI npm packages / projects and downstream developer or build environments.
What is the defender action?	Identify affected package versions, rotate exposed secrets, review CI/CD runner telemetry, and monitor npm postinstall execution.
Source window	Primary reporting was published within the last 7 days; this run expanded beyond 72 hours because high-confidence sector-specific items were limited.

3. Source Review & Confidence

Source	Contribution	Confidence
Microsoft Security Blog	Primary report, attribution, defender framing, Defender XDR hunting orientation	High
The Hacker News / industry reporting	Package count and supply-chain incident context based on vendor reports	Medium
Secondary vendor/security-news coverage	Corroborates broad developer-supply-chain risk and public awareness	Medium

Confidence is medium-high: the intrusion narrative depends primarily on Microsoft reporting. This report does not introduce new IOCs beyond observable patterns and package-scope indicators that defenders should validate internally.

4. Threat Actor / Campaign Overview

Sapphire Sleet activity has been associated by Microsoft with financially motivated developer targeting and credential theft. In this case, the notable tradecraft is abusing trusted package distribution and npm lifecycle behavior. Postinstall payload execution is especially dangerous because developers and CI/CD systems often run dependency installation with broad environment-variable and credential access.

5. Tactics, Techniques, and Procedures

Tactic	Technique	Observed / Assessed Behavior
Initial Access	Supply Chain Compromise	Poisoned npm packages in the Mastra ecosystem
Execution	Command and Scripting Interpreter	npm postinstall lifecycle script execution
Credential Access	Unsecured Credentials	Potential collection of API keys, tokens, cloud credentials, and wallet material from developer/build environments
Exfiltration	Exfiltration Over Web Service	Assessed risk for outbound credential staging; validate against local telemetry

6. Infrastructure and Indicators

Indicator Type	Value / Pattern	Notes
Package scope	@mastra/*	Treat as a starting point for package inventory; validate exact versions from package-lock/yarn/pnpm lockfiles
Lifecycle behavior	postinstall script execution during npm install	High-signal behavior when newly introduced or network-active
Host role	Developer workstation or CI/CD runner	Highest value environments for secrets exposure

No hard network IOCs were validated during this run. `iocs.csv` therefore records patterns and analyst placeholders, not confirmed malicious infrastructure.

7. Aviation / Aerospace Sector Impact

Segment	Operational Relevance	Likely Exposure Path	Defender Action
Aerospace software engineering	AI-agent and automation frameworks may be tested in engineering productivity tooling	Developer package installation and CI/CD builds	Inventory @mastra dependencies and freeze builds until lockfiles are reviewed
Airlines / airports digital teams	Internal automation, customer-service bots, or ops tooling may consume npm AI packages	SaaS prototypes, innovation labs, contractor repos	Search repos and CI logs for Mastra packages and postinstall execution
Defense aerospace suppliers	Build runners may hold signing keys, cloud tokens, artifact-registry credentials	Contractor and subcontractor development pipelines	Rotate secrets if exposure is plausible; review artifact provenance

8. Detection Engineering

8.1. Detection Logic Summary

Signal	Telemetry Source	Analytic Goal
--------	------------------	---------------

npm postinstall spawning shell/network tooling	EDR process telemetry	Detect suspicious lifecycle payload execution
CI runner outbound connections after dependency install	Proxy/firewall/build logs	Identify credential exfiltration windows
New @mastra package versions in lockfiles	SCM/package inventory	Scope exposure and affected projects
<pre> title: NPM Postinstall Script Spawns Suspicious Child Process status: experimental logsource: category: process_creation product: windows detection: parent: ParentImage endswith: - \\npm.cmd - \\node.exe selection: CommandLine contains: - postinstall - curl - wget - powershell - bash - node -e condition: parent and selection level: high </pre>		
<pre> DeviceProcessEvents where Timestamp > ago(30d) where InitiatingProcessFileName in~ ("npm.cmd", "npm", "node.exe", "node") where ProcessCommandLine has_any ("postinstall", "curl", "wget", "powershell", "bash", "node -e", "@mastra") project Timestamp, DeviceName, AccountName, InitiatingProcessCommandLine, FileName, ProcessCommandLine </pre>		

9. Threat Hunting

9.1. Hunt Hypothesis

A poisoned npm package executed in developer or CI environments will show npm lifecycle execution followed by unusual child processes, outbound connections, or secret-file access.

<pre> DeviceFileEvents where Timestamp > ago(30d) where FolderPath has_any ("package-lock.json", "yarn.lock", "pnpm-lock.yaml") where PreviousFileName has_any ("package-lock.json", "yarn.lock", "pnpm-lock.yaml") or FileName has_any ("package-lock.json", "yarn.lock", "pnpm-lock.yaml") join kind=inner (DeviceProcessEvents where ProcessCommandLine has "@mastra" or InitiatingProcessCommandLine has "npm install") on DeviceId project Timestamp, DeviceName, FolderPath, FileName, ProcessCommandLine, AccountName </pre>
<pre> index=edr (process_name=npm OR process_name=node) ("postinstall" OR "@mastra" OR "node -e" OR "curl" OR "wget") stats earliest(_time) as first_seen latest(_time) as last_seen values(process_cmdline) as cmds by host user parent_process_name process_name </pre>

10. Incident Response and Containment

Priority	Action	Rationale
Critical	Freeze builds that installed affected Mastra	Prevents repeated postinstall execution

	packages until reviewed	
Critical	Rotate tokens available to impacted developer and CI environments	Assume environment variables and local secrets may have been exposed
High	Compare lockfiles to known-good commits and artifact timestamps	Identifies affected projects and build outputs
High	Rebuild artifacts from clean runners after dependency pinning	Restores provenance

11. MITRE ATT&CK Mapping

Technique	Name	Rationale
T1195.002	Compromise Software Supply Chain	Poisoned npm packages
T1059	Command and Scripting Interpreter	Postinstall payload execution
T1552	Unsecured Credentials	Developer/build environments often contain tokens and keys
T1041	Exfiltration Over C2 Channel	Hunt for outbound transfer after install events

12. Recommended Actions

1. Search all repositories for `@mastra/` package references and identify exact installed versions.
2. Review CI/CD logs for npm install windows and unexpected outbound traffic.
3. Rotate package-registry, GitHub/GitLab, cloud, signing, and deployment secrets available to affected environments.
4. Add blocking controls for package lifecycle scripts where feasible (`npm ci --ignore-scripts` for validation stages, with compensating review when scripts are required).
5. Educate development teams that AI-package ecosystems are now high-value supply-chain targets.

13. References

- [1] Microsoft Security Blog: From package to postinstall payload: Inside the Mastra npm supply chain compromise by Sapphire Sleet
- [2] The Hacker News: Mastra npm packages compromised via hijacked contributor account
- [3] Public secondary coverage of Sapphire Sleet / Mastra npm supply-chain compromise